

Deep Learning

Vazgen Mikayelyan

July 3, 2021



Variational Autoencoders

- Can we generate content with autoencoders?

Variational Autoencoders

- Can we generate content with autoencoders?
- Variational autoencoder can be defined as being an autoencoder whose training is regularised to avoid overfitting and ensure that the latent space has good properties that enable generative process.

Variational Autoencoders

- Can we generate content with autoencoders?
- Variational autoencoder can be defined as being an autoencoder whose training is regularised to avoid overfitting and ensure that the latent space has good properties that enable generative process.
- Instead of encoding an input as a single point, we encode it as a distribution over the latent space.

Variational Autoencoders

- Denote distribution of latent representations by $p(z)$.

Variational Autoencoders

- Denote distribution of latent representations by $p(z)$.
- Denote distribution of ideal encoder by $p(z|x)$.

Variational Autoencoders

- Denote distribution of latent representations by $p(z)$.
- Denote distribution of ideal encoder by $p(z|x)$.
- We want to find a neural network $q_w(z|x)$ such that

$$q_w(z|x) \approx p(z|x).$$

Variational Autoencoders

- Denote distribution of latent representations by $p(z)$.
- Denote distribution of ideal encoder by $p(z|x)$.
- We want to find a neural network $q_w(z|x)$ such that

$$q_w(z|x) \approx p(z|x).$$

- Let minimize the function

$$L(w) = KL(q_w(z|x) || p(z|x)).$$

Variational Autoencoders

Note that

$$KL(q_w(z|x) || p(z|x)) = \mathbb{E}_{q_w(z|x)} \left[\log \frac{q_w(z|x)}{p(z|x)} \right]$$

Variational Autoencoders

Note that

$$\begin{aligned} KL(q_w(z|x) || p(z|x)) &= \mathbb{E}_{q_w(z|x)} \left[\log \frac{q_w(z|x)}{p(z|x)} \right] \\ &= \mathbb{E}_{q_w(z|x)} [\log q_w(z|x)] - \mathbb{E}_{q_w(z|x)} [\log p(z|x)] \end{aligned}$$

Variational Autoencoders

Note that

$$\begin{aligned} KL(q_w(z|x) || p(z|x)) &= \mathbb{E}_{q_w(z|x)} \left[\log \frac{q_w(z|x)}{p(z|x)} \right] \\ &= \mathbb{E}_{q_w(z|x)} [\log q_w(z|x)] - \mathbb{E}_{q_w(z|x)} [\log p(z|x)] \\ &= \mathbb{E}_{q_w(z|x)} [\log q_w(z|x)] - \mathbb{E}_{q_w(z|x)} \left[\log \frac{p(x|z)p(z)}{p(x)} \right] \end{aligned}$$

Variational Autoencoders

Note that

$$\begin{aligned}KL(q_w(z|x) || p(z|x)) &= \mathbb{E}_{q_w(z|x)} \left[\log \frac{q_w(z|x)}{p(z|x)} \right] \\&= \mathbb{E}_{q_w(z|x)} [\log q_w(z|x)] - \mathbb{E}_{q_w(z|x)} [\log p(z|x)] \\&= \mathbb{E}_{q_w(z|x)} [\log q_w(z|x)] - \mathbb{E}_{q_w(z|x)} \left[\log \frac{p(x|z)p(z)}{p(x)} \right] \\&= \mathbb{E}_{q_w(z|x)} \left[\log \frac{q_w(z|x)}{p(z)} \right] - \mathbb{E}_{q_w(z|x)} [\log p(x|z)] + \mathbb{E}_{q_w(z|x)} [\log p(x)]\end{aligned}$$

Variational Autoencoders

Note that

$$\begin{aligned}KL(q_w(z|x) || p(z|x)) &= \mathbb{E}_{q_w(z|x)} \left[\log \frac{q_w(z|x)}{p(z|x)} \right] \\&= \mathbb{E}_{q_w(z|x)} [\log q_w(z|x)] - \mathbb{E}_{q_w(z|x)} [\log p(z|x)] \\&= \mathbb{E}_{q_w(z|x)} [\log q_w(z|x)] - \mathbb{E}_{q_w(z|x)} \left[\log \frac{p(x|z)p(z)}{p(x)} \right] \\&= \mathbb{E}_{q_w(z|x)} \left[\log \frac{q_w(z|x)}{p(z)} \right] - \mathbb{E}_{q_w(z|x)} [\log p(x|z)] + \mathbb{E}_{q_w(z|x)} [\log p(x)] \\&= KL(q_w(z|x) || p(z)) - \mathbb{E}_{q_w(z|x)} [\log p(x|z)] + \log p(x).\end{aligned}$$

Variational Autoencoders

Note that

$$\begin{aligned}KL(q_w(z|x) || p(z|x)) &= \mathbb{E}_{q_w(z|x)} \left[\log \frac{q_w(z|x)}{p(z|x)} \right] \\&= \mathbb{E}_{q_w(z|x)} [\log q_w(z|x)] - \mathbb{E}_{q_w(z|x)} [\log p(z|x)] \\&= \mathbb{E}_{q_w(z|x)} [\log q_w(z|x)] - \mathbb{E}_{q_w(z|x)} \left[\log \frac{p(x|z)p(z)}{p(x)} \right] \\&= \mathbb{E}_{q_w(z|x)} \left[\log \frac{q_w(z|x)}{p(z)} \right] - \mathbb{E}_{q_w(z|x)} [\log p(x|z)] + \mathbb{E}_{q_w(z|x)} [\log p(x)] \\&= KL(q_w(z|x) || p(z)) - \mathbb{E}_{q_w(z|x)} [\log p(x|z)] + \log p(x).\end{aligned}$$

So we have to model the distribution $p(x|z)$ too, which will be our decoder:

Variational Autoencoders

Note that

$$\begin{aligned}KL(q_w(z|x) || p(z|x)) &= \mathbb{E}_{q_w(z|x)} \left[\log \frac{q_w(z|x)}{p(z|x)} \right] \\&= \mathbb{E}_{q_w(z|x)} [\log q_w(z|x)] - \mathbb{E}_{q_w(z|x)} [\log p(z|x)] \\&= \mathbb{E}_{q_w(z|x)} [\log q_w(z|x)] - \mathbb{E}_{q_w(z|x)} \left[\log \frac{p(x|z)p(z)}{p(x)} \right] \\&= \mathbb{E}_{q_w(z|x)} \left[\log \frac{q_w(z|x)}{p(z)} \right] - \mathbb{E}_{q_w(z|x)} [\log p(x|z)] + \mathbb{E}_{q_w(z|x)} [\log p(x)] \\&= KL(q_w(z|x) || p(z)) - \mathbb{E}_{q_w(z|x)} [\log p(x|z)] + \log p(x).\end{aligned}$$

So we have to model the distribution $p(x|z)$ too, which will be our decoder:

$$\operatorname{argmin}_{w, w'} (KL(q_w(z|x) || p(z)) + \mathbb{E}_{q_w(z|x)} [\|x - f_{w'}(z)\|^2])$$

Problem: how to evaluate the term $KL(q_w(z|x) || p(z))$?

Problem: how to evaluate the term $KL(q_w(z|x) || p(z))$?

Solution: we will assume that $q_w = \mathcal{N}(\mu, \Sigma)$ and $p = \mathcal{N}(0, I)$.

Variational Autoencoders

The model is then trained as follows:

- first, the input is encoded as distribution over the latent space,

Variational Autoencoders

The model is then trained as follows:

- first, the input is encoded as distribution over the latent space,
- second, a point from the latent space is sampled from that distribution,

Variational Autoencoders

The model is then trained as follows:

- first, the input is encoded as distribution over the latent space,
- second, a point from the latent space is sampled from that distribution,
- third, the sampled point is decoded and the reconstruction error can be computed,

Variational Autoencoders

The model is then trained as follows:

- first, the input is encoded as distribution over the latent space,
- second, a point from the latent space is sampled from that distribution,
- third, the sampled point is decoded and the reconstruction error can be computed,
- finally, the reconstruction error is backpropagated through the network.

Variational Autoencoders

The model is then trained as follows:

- first, the input is encoded as distribution over the latent space,
- second, a point from the latent space is sampled from that distribution,
- third, the sampled point is decoded and the reconstruction error can be computed,
- finally, the reconstruction error is backpropagated through the network.

Assumptions

- In practice, the encoded distributions are chosen to be normal so that the encoder can be trained to return the mean and the covariance matrix that describe these Gaussians.

Variational Autoencoders

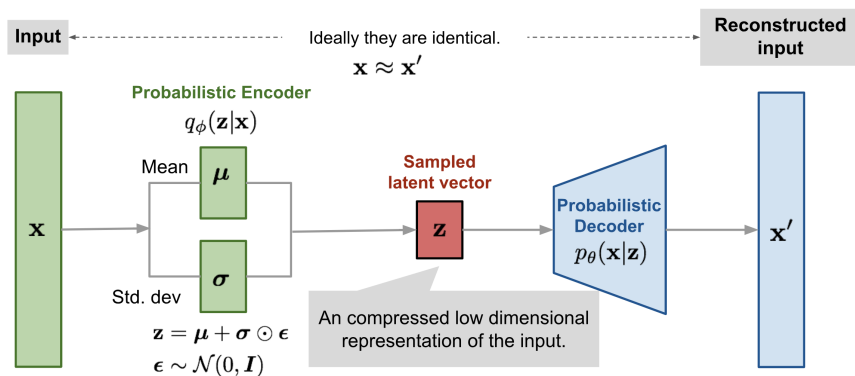
The model is then trained as follows:

- first, the input is encoded as distribution over the latent space,
- second, a point from the latent space is sampled from that distribution,
- third, the sampled point is decoded and the reconstruction error can be computed,
- finally, the reconstruction error is backpropagated through the network.

Assumptions

- In practice, the encoded distributions are chosen to be normal so that the encoder can be trained to return the mean and the covariance matrix that describe these Gaussians.
- The distributions returned by the encoder are enforced to be close to a standard normal distribution.

Variational Autoencoders



Variational Autoencoders

Let f is our encoder, g is the decoder and D is our training dataset. In this case we will minimize the following loss function

$$\sum_{x \in D} L(x, g(f(x))) + \lambda KL(N(\mu, \Sigma) || N(0, I)).$$

1 Generative Adversarial Networks

- GANs are neural networks that generate synthetic data given certain input data.

- GANs are neural networks that generate synthetic data given certain input data.
- GANs consist of two models:

- GANs are neural networks that generate synthetic data given certain input data.
- GANs consist of two models:
 - **Discriminator**
takes samples of true and generated data and that try to classify them as well as possible.

- GANs are neural networks that generate synthetic data given certain input data.
- GANs consist of two models:
 - **Discriminator**
takes samples of true and generated data and that try to classify them as well as possible.
 - **Generator**
trained to fool the discriminator as much as possible by generating fake data.

Different types of GANs

- Simple GANs

Different types of GANs

- Simple GANs
- Deep Convolutional GANs (DCGANs)

Different types of GANs

- Simple GANs
- Deep Convolutional GANs (DCGANs)
- Conditional GANs (cGANs)

Different types of GANs

- Simple GANs
- Deep Convolutional GANs (DCGANs)
- Conditional GANs (cGANs)
- CycleGANs

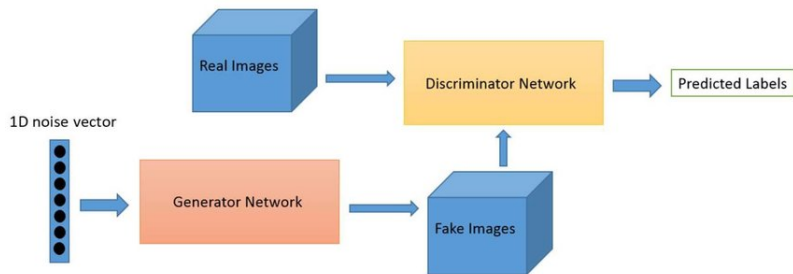
Different types of GANs

- Simple GANs
- Deep Convolutional GANs (DCGANs)
- Conditional GANs (cGANs)
- CycleGANs
- Discover Cross-Domain Relations with Generative Adversarial Networks (Disco GANs)

Different types of GANs

- Simple GANs
- Deep Convolutional GANs (DCGANs)
- Conditional GANs (cGANs)
- CycleGANs
- Discover Cross-Domain Relations with Generative Adversarial Networks (Disco GANs)
- Wasserstein GANs (WGANs)

Simple GANs



The Loss Function

Let p_z and p_{data} be respectively the distributions of input noise and our data and let D and G be respectively discriminator and generator.

The Loss Function

Let p_z and p_{data} be respectively the distributions of input noise and our data and let D and G be respectively discriminator and generator. We will do the following optimization

$$\min_G \max_D (\mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log (1 - D(G(z)))])$$

Optimization Algorithm for GANs

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Sample minibatch of m examples $\{x^{(1)}, \dots, x^{(m)}\}$ from data generating distribution $p_{\text{data}}(x)$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(x^{(i)}) + \log (1 - D(G(z^{(i)}))) \right].$$

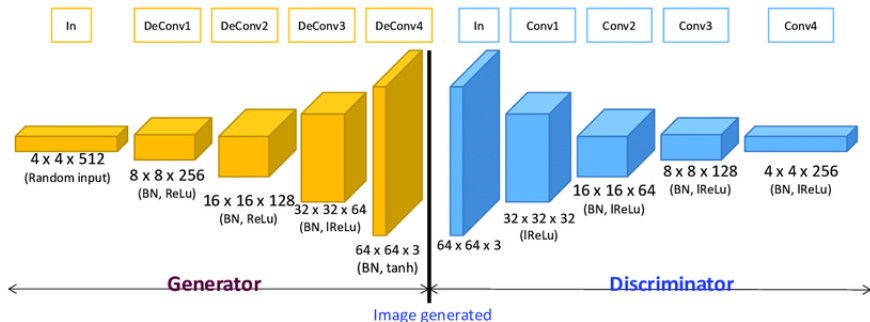
end for

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(z^{(i)}))).$$

end for

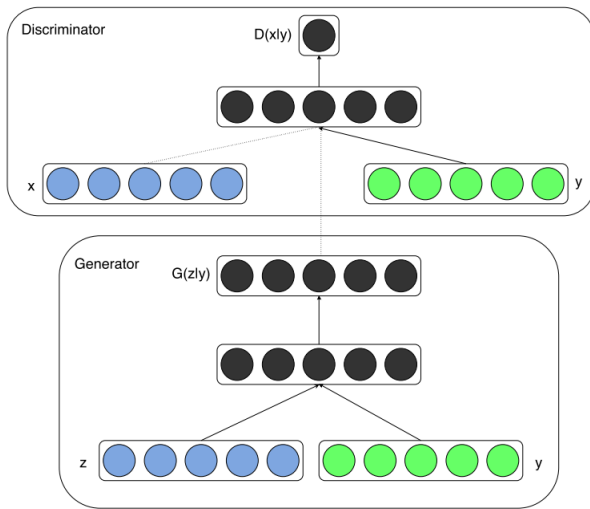
DCGANs



These GANs use extra label information and result in better quality images and are able to control how generated images will look.

These GANs use extra label information and result in better quality images and are able to control how generated images will look. In this case we will do the following optimization

$$\min_G \max_D (\mathbb{E}_{x \sim p_{data}} [\log D(x|y)] + \mathbb{E}_{z \sim p_z} [\log (1 - D(G(z|y)))])$$



- Image-to-image translation is a class of vision and graphics problems where the goal is to learn the mapping between an input image and an output image using a training set of aligned image pairs.

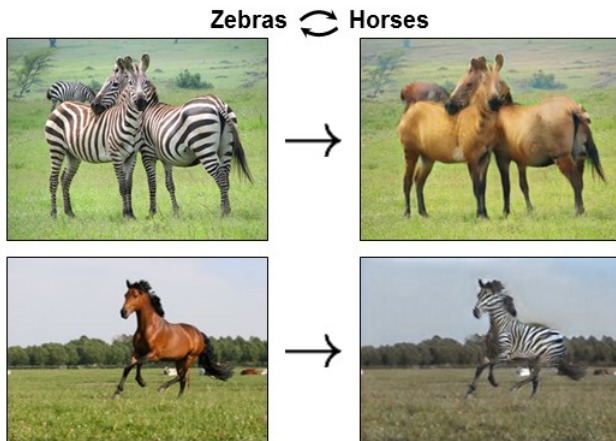
- Image-to-image translation is a class of vision and graphics problems where the goal is to learn the mapping between an input image and an output image using a training set of aligned image pairs.
- However, for many tasks, paired training data will not be available.

- Image-to-image translation is a class of vision and graphics problems where the goal is to learn the mapping between an input image and an output image using a training set of aligned image pairs.
- However, for many tasks, paired training data will not be available.
- So our task is to learn how to translate an image from a source domain X to a target domain Y in the absence of paired examples.

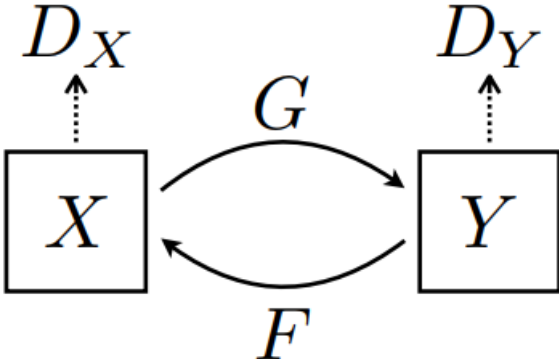
- Image-to-image translation is a class of vision and graphics problems where the goal is to learn the mapping between an input image and an output image using a training set of aligned image pairs.
- However, for many tasks, paired training data will not be available.
- So our task is to learn how to translate an image from a source domain X to a target domain Y in the absence of paired examples.
- Our goal is to learn a mapping $G : X \rightarrow Y$ such that the distribution of images from $G(X)$ is indistinguishable from the distribution Y .

- Image-to-image translation is a class of vision and graphics problems where the goal is to learn the mapping between an input image and an output image using a training set of aligned image pairs.
- However, for many tasks, paired training data will not be available.
- So our task is to learn how to translate an image from a source domain X to a target domain Y in the absence of paired examples.
- Our goal is to learn a mapping $G : X \rightarrow Y$ such that the distribution of images from $G(X)$ is indistinguishable from the distribution Y .
- We will couple it with an inverse mapping $F : Y \rightarrow X$ and introduce a cycle consistency loss to enforce $F(G(X)) \approx X$.

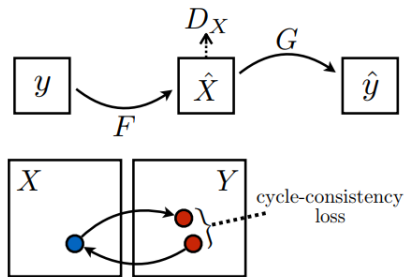
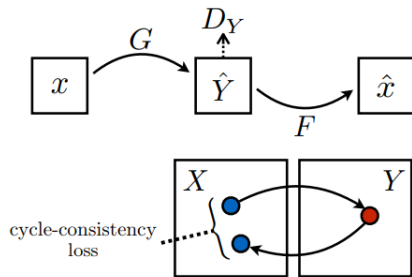
CycleGANs



CycleGANs



CycleGANs



CycleGANs

Our loss function will be the following

$$L(G, F, D_X, D_Y) = L_{GAN}(G, D_Y, X, Y) + L_{GAN}(F, D_X, Y, X) + \lambda L_{cyc}(G, F),$$

CycleGANs

Our loss function will be the following

$$L(G, F, D_X, D_Y) = L_{GAN}(G, D_Y, X, Y) + L_{GAN}(F, D_X, Y, X) + \lambda L_{cyc}(G, F),$$

where

$$\begin{aligned} & L_{GAN}(G, D_Y, X, Y) \\ &= \mathbb{E}_{y \sim p_{data}(y)} [\log D_Y(y)] + \mathbb{E}_{x \sim p_{data}(x)} [\log (1 - D_Y(G(x)))] \end{aligned}$$

CycleGANs

Our loss function will be the following

$$L(G, F, D_X, D_Y) = L_{GAN}(G, D_Y, X, Y) + L_{GAN}(F, D_X, Y, X) + \lambda L_{cyc}(G, F),$$

where

$$\begin{aligned} & L_{GAN}(G, D_Y, X, Y) \\ &= \mathbb{E}_{y \sim p_{data}(y)} [\log D_Y(y)] + \mathbb{E}_{x \sim p_{data}(x)} [\log (1 - D_Y(G(x)))] \end{aligned}$$

$$\begin{aligned} & L_{GAN}(F, D_X, Y, X) \\ &= \mathbb{E}_{x \sim p_{data}(x)} [\log D_X(x)] + \mathbb{E}_{y \sim p_{data}(y)} [\log (1 - D_X(F(y)))] \end{aligned}$$

CycleGANs

Our loss function will be the following

$$L(G, F, D_X, D_Y) = L_{GAN}(G, D_Y, X, Y) + L_{GAN}(F, D_X, Y, X) + \lambda L_{cyc}(G, F),$$

where

$$\begin{aligned} & L_{GAN}(G, D_Y, X, Y) \\ &= \mathbb{E}_{y \sim p_{data}(y)} [\log D_Y(y)] + \mathbb{E}_{x \sim p_{data}(x)} [\log (1 - D_Y(G(x)))] \end{aligned}$$

$$\begin{aligned} & L_{GAN}(F, D_X, Y, X) \\ &= \mathbb{E}_{x \sim p_{data}(x)} [\log D_X(x)] + \mathbb{E}_{y \sim p_{data}(y)} [\log (1 - D_X(F(y)))] \end{aligned}$$

$$L_{cyc}(G, F) = \mathbb{E}_{x \sim p_{data}(x)} [\|F(G(x)) - x\|_1] + \mathbb{E}_{y \sim p_{data}(y)} [\|G(F(y)) - y\|_1]$$

CycleGANs

Our loss function will be the following

$$L(G, F, D_X, D_Y) = L_{GAN}(G, D_Y, X, Y) + L_{GAN}(F, D_X, Y, X) + \lambda L_{cyc}(G, F),$$

where

$$\begin{aligned} & L_{GAN}(G, D_Y, X, Y) \\ &= \mathbb{E}_{y \sim p_{data}(y)} [\log D_Y(y)] + \mathbb{E}_{x \sim p_{data}(x)} [\log (1 - D_Y(G(x)))] \end{aligned}$$

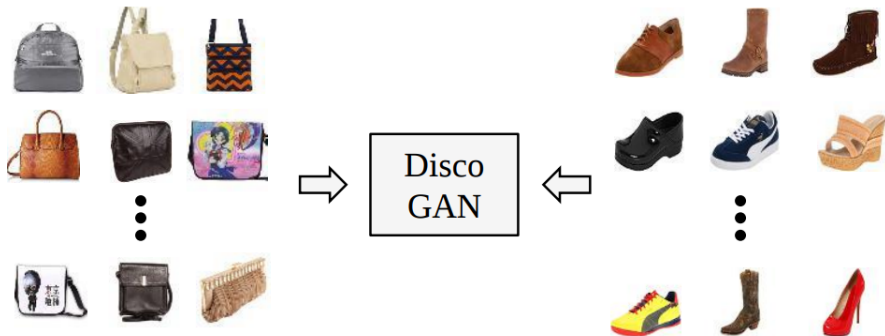
$$\begin{aligned} & L_{GAN}(F, D_X, Y, X) \\ &= \mathbb{E}_{x \sim p_{data}(x)} [\log D_X(x)] + \mathbb{E}_{y \sim p_{data}(y)} [\log (1 - D_X(F(y)))] \end{aligned}$$

$$L_{cyc}(G, F) = \mathbb{E}_{x \sim p_{data}(x)} [\|F(G(x)) - x\|_1] + \mathbb{E}_{y \sim p_{data}(y)} [\|G(F(y)) - y\|_1]$$

We aim to solve

$$G^*, F^* = \arg \min_{G, F} \max_{D_X, D_Y} L(G, F, D_X, D_Y)$$

DISCO GAN



DISCO GAN

INPUT



OUTPUT



DISCO GAN

INPUT



OUTPUT



$$L_{\delta}(y, f(x)) = \begin{cases} \frac{1}{2} (y - f(x))^2, & \text{for } |y - f(x)| \leq \delta \\ \delta |y - f(x)| - \frac{1}{2} \delta^2, & \text{otherwise.} \end{cases}$$